

FINAL ASSIGNMENT : Schedule search

Execution-Time Statistics

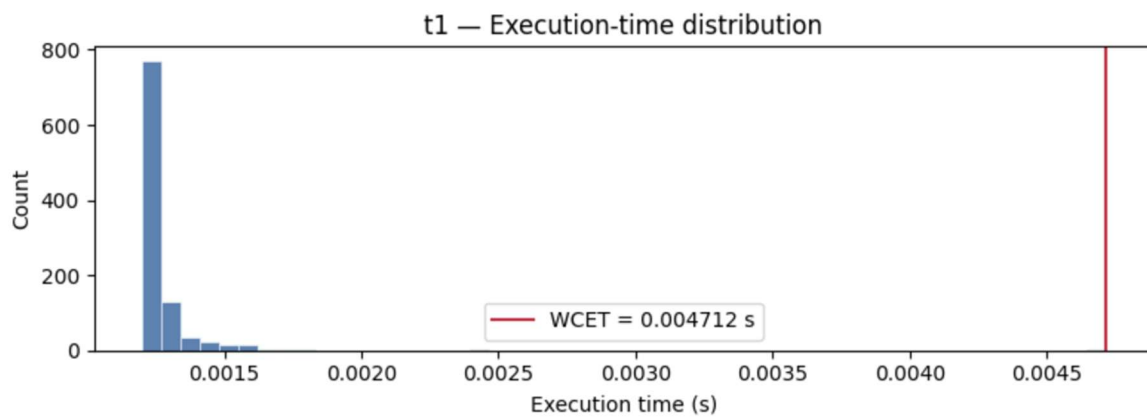
In order to measure the execution time of task τ_1 (multiplication of two large randomly generated numbers), the task was implemented in C and executed 1000 times.

Execution times were recorded in a CSV file.

Statistics from 1000 executions:

- Min: 0.001200 s
- Q1 (25th percentile): 0.001219 s
- Q2 — Median: 0.001230 s
- Q3 (75th percentile): 0.001266 s
- Max — WCET (C_1): 0.004712 s

The Worst-Case Execution Time (WCET) $C_1 = 0.004712$ s is used in all subsequent scheduling computations.



Task Set Definition

implicit deadlines $D_i = T_i$

Task	C	T	D

τ_1	0.004712	10	10
τ_2	3.000000	10	10
τ_3	2.000000	20	20
τ_4	2.000000	20	20
τ_5	2.000000	40	40
τ_6	2.000000	40	40
τ_7	3.000000	80	80

Total processor utilisation $U = \sum C_i/T_i = 0.637971$

Schedulability Analysis

For the calculation we have made the following assumptions :

- Implicit deadlines: $D_i = T_i$ for all tasks
- Tasks are independent and periodic
- Non-preemptive scheduling: once a job starts it runs to completion
- Hyperperiod $H = \text{LCM}(10, 20, 40, 80) = 80$ time units
- Necessary condition: processor utilisation $U \leq 1$

So we have :

$$U = 0.004712/10 + 3/10 + 2/20 + 2/20 + 2/40 + 2/40 + 3/80 = 0.637971$$

Since **$U = 0.638 \leq 1$** , the task set is schedulable.

Processor idle-time budget over hyperperiod:

$$I = H \times (1 - U) = 80 \times (1 - 0.637971) = 28.9623 \text{ time units}$$

Non-Preemptive Schedule Search

We have made the following assumptions :

- Job-level scheduling over one hyperperiod ($H = 80$)
- Priority assignment: tasks have fixed static priority
- Objective: minimise total waiting time
- Note that: processor idle time $I = H \cdot (1 - U)$ is constant for any valid schedule

For the complexity :

Exhaustive search over all possible priority orderings:

- Generate all $n!$ permutations ($n = 7$ tasks)
- Simulate each permutation over H time units
- Track valid schedules and find minimum waiting time

$$\text{Complexity} = O(n! \times J)$$

where n = number of tasks, J = total jobs in hyperperiod

For our exemple:

- $n = 7 \Rightarrow 7! = 5,040$ permutations
- $J = 29$ jobs total in hyperperiod
- Total operations: $5,040 \times 29 \approx 146,000$

So the results are :

```
Valid schedules : 1824 / 5040
Optimal order   :  $\tau_1 > \tau_3 > \tau_4 > \tau_5 > \tau_2 > \tau_6 > \tau_7$ 
Min total wait  : 72.1225 time units
```

- Valid schedules: 1,824 / 5,040 (36.2%)
- Optimal order: $\tau_1 > \tau_3 > \tau_4 > \tau_5 > \tau_2 > \tau_6 > \tau_7$
- Minimum total waiting time: 72.1225 time units

Response-Time Analysis

For each job: $R_i = \text{finish}_i - \text{arrival}_i$

Schedulability requires: $R_i \leq D_i = T_i$ for every job

All 29 jobs over the hyperperiod were verified by the code, you can check it in the folder.

Result: All 29 jobs meet their deadline. Missed deadlines = 0.

Processor Idle-Time Verification

Theoretical idle time: $I = H \times (1 - U) = 28.9623$ time units

```
Theoretical idle time  H·(1-U)          = 28.9623
Optimal schedule idle time          = 28.9623
```

Measured idle time matches theory exactly.

We can note that Idle time is constant (28.9623) for all valid schedules, not just the optimal one. Minimising waiting time $\neq$ maximising idle time.

Schedule with t_5 Allowed to Miss Its Deadline

The same search is repeated.

```
Valid schedules          : 1824 / 5040
Optimal order            :  $\tau_1 > \tau_3 > \tau_4 > \tau_5 > \tau_2 > \tau_6 > \tau_7$ 
Min total wait           : 72.1225
t5 deadline misses       : 0
Idle time                 : 28.9623
```

Result: identical to base case, same optimal order, same waiting time (72.1225), 0 deadline misses for τ_5 .

The explanation is $C_1 = 0.004712$ s is extremely small compared to other tasks (2–3 s), so t_5 never misses in any feasible schedule.

Gantt Chart

Both schedules (optimal and t_5 -tolerant) are identical. Note that t_1 is barely visible due to its very small execution time ($C_1 = 0.0047$ s).

